

24+25年的重复点



Author: 潜伏

写作日期: 2026-06-06

主题: Java 期末复习重复考点整理

- Java 语言特征：不支持指针
- 一个类可以实现(implements)多个接口,一个接口也可以继承(extends)多个结果
- 在父类和类中，如果有同名变量，如果不加任何修饰，就是引用类本身的变量，如果加了 super 关键字就是父类变量
- 白黑盒测试相关概念

常见黑盒测试

- 等价类划分
- 边界值分析
- 场景法

常见白盒测试

白盒覆盖标准从弱到强有：

- 语句覆盖 设计一个样例执行一遍
 - 判定覆盖 / 分支覆盖 每个判定的 true 和 false 分支都至少执行一次
 - 条件覆盖 每个简单条件的 true 和 false 都至少出现一次
 - 条件组合覆盖 每个判定中所有简单条件的所有取值组合都至少执行一次
 - 路径覆盖 程序中所有可能执行路径至少执行一次
- swing 相关概念。

swing 界面上的东西分为两类： 组件与容器

组件例如：

- JButton 按钮
- JLabel 标签
- JTextField 文本框
- JCheckBox 复选框

- JTable 表格

容器例如：

- JFrame
- JDialog
- JPanel
- JScrollPane

容器之所以是容器，是因为可以放其他组件，不过容器本身也是组件，可以被人装

JFrame, JDialog, JWindow, JApplet 属于重量级容器/顶层容器，可以独立显示为一个窗口，且不可以放到其他容器

- 线程的概念

- start 与 run 的区别，run方法之后被当作一个普通的方法，而不会启动新的线程，而 start 方法会创建新的线程
- 优先级高的线程不一定比优先级低的线程优先执行，优先级高的线程只是获得 CPU 执行时间片的概率更大，但绝不是绝对优先或保证一定先执行完毕。
- 计时器不是守护线程而是用户线程

- Java 语言编写网络通信程序一般位于 TCP/IP 的哪一个层级？应用层 (两年原题)

- 面向对象语言的三大特性：多态，封装，继承

- 泛型通配符<? T>， extends 就是子类， super 就是父类

- MVC 设计模式相关概念。MVC 设计模式，包括**模型层,视图层,控制器层**。模型与视图之间通过 **观察者** 模式交互。

- TCP/IP 是最常用的网络协议。它基于 字节 流传输方式。(两年原题)

- ArrayList LinkedList HashSet HashMap 的辨析

- ArrayList 适合访问，不适合插入删除
- LinkedList 插入/删除快，访问慢
- HashSet 无序且唯一的集合，不允许有重复元素存在
- HashMap key-value 存储
- ArrayList LinkedList 实现 List接口 HashSet 实现 Set接口，Set和 List 都继承了 Collection 接口，HashMap实现 Map接口

- 按照目的来划分，设计模式可以分为**创建型**模式、**结构型**模式和**行为型**模式。

- 工厂方法，抽象工厂，单例模式都属于创建型
- 策略模式，观察者模式，迭代器模式属于行为型模式

- super 和 this 的辨析

- 抽象类和接口的异同(两年原题):

- 都不能实例化
- 都包含抽象方法
- 都可以用来实现多态
- 一个类只能继承一个抽象类，但可以实现多接口
- 抽象类的变量很自由，接口只能有静态变量

- UML 类图
 - -=private ~-=default #-=protected +=public
 - 三角实=继承
 - 三角虚=实现
 - 箭头实 A与B有关系
 - 箭头虚 A依赖B
 - 实心菱形 AB 同生共死
 - 空心菱形 A没B可以有
- 生产者和消费者模式有点
 - 解耦，降低耦合度
 - 支持并发和异步处理
 - 削峰填谷，平衡生产数据速度和消费数据速度的差异
- 单例模式的饿汉和懒汉，优缺点与代码实现，是否可以抵抗反射
饿汉式：

```
public class Singleton {  
    // 1. 声明并实例化一个私有的静态成员变量（饿汉式的关键：类加载时即创建）  
    private static Singleton instance = new Singleton();  
  
    // 2. 将构造方法私有化，防止外部使用 new 关键字创建新的实例  
    private Singleton() {  
    }  
  
    // 3. 提供一个全局访问点（公共静态方法），供外部获取该唯一实例  
    public static Singleton getInstance() {  
        return instance;  
    }  
}
```

懒汉式：

```

public class Singleton {
    // 1. 声明一个私有的静态变量，初始为 null（不立刻实例化）
    private static Singleton instance = null;

    // 2. 构造方法私有化，防止外部 new
    private Singleton() {
    }

    // 3. 提供全局访问点
    public static Singleton getInstance() {
        // 如果实例尚未创建，则创建一个新实例
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}

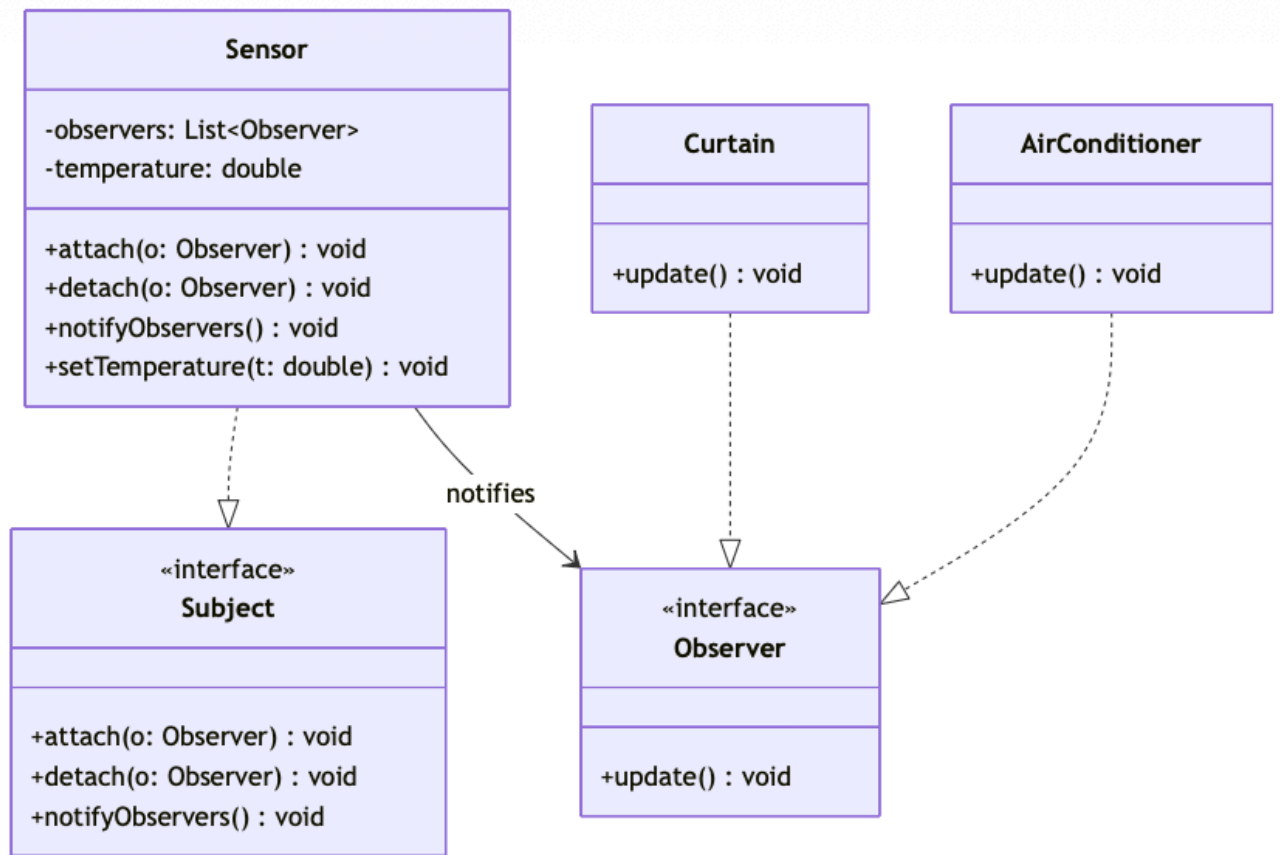
```

- 模版模式：

- 概念：定义一个操作中的算法骨架，将一些步骤延迟到子类中使得子类可以不改变一个算法的结构即可重新定义该算法的某些特定步骤
- 优缺点：
 - 符合开闭原则
 - 易于复用

- 观察者模式

- 概念：定义对象间的一种一对多依赖关系，每当一个对象改变的时候，所有依赖它的对象都会自动得到通知并更新
- 优缺点
 - 降低耦合度
 - 符合开闭原则
 - 触发循环依赖导致系统崩溃
 - 性能和时间消耗
- UML 类图



- 策略模式的目的，如何实现的，策略模式的三个角色
 - 目的：符合开闭原则；消除冗长的 if-else 或者 switch-case
 - 策略模式的三个角色：抽象策略角色；具体策略角色；上下文角色。
 - UML 类图

